

Phase 5 — Backend Architecture & System Design

Full roadmap for the current phase, with progress so far

This phase covers how a backend is actually built and shaped for real use — not just "routes that work," but an API and system that behaves correctly, fails safely, and can grow. Work started at basic API/routing fundamentals to build a solid base before the more advanced system-design topics below.

Covered So Far

- What an API is, and the request/response cycle
- Node vs Express — who actually creates and hosts the server
- Routing — URL + HTTP method mapped to a handler function
- Full CRUD: GET (list & single), POST, PUT, DELETE
- Route parameters (`:id`) and reading a request body (`express.json()`)
- Correct status codes: 200, 201, 400, 404
- Fixed a real duplicate-id bug (counter instead of array length)
- Input validation — missing, empty, and wrong-type fields
- Fixed a real crash bug from unvalidated input types
- Error handling — sync vs async errors, a global error-handling middleware
- Middleware concept and the request-handling chain (`next()`)

In Progress

- Replacing the in-memory array with a real, persistent database (SQLite)
- Understanding why persistence matters — data surviving a server restart

Still Ahead in This Phase

API design, refined

- Resource-based URLs done rigorously (nouns, not verbs)
- Idempotency — why GET/PUT/DELETE should be safe to repeat, POST usually isn't
- Idempotency keys — letting a client safely retry a POST without duplicating it

- Full status-code vocabulary — 401 vs 403, 409 conflicts, 422 semantic errors
- Pagination — offset pagination and why it breaks under concurrent writes; cursor-based pagination as the stable alternative
- API versioning — introducing breaking changes without breaking existing clients
- REST vs gRPC — when binary/typed contracts are worth losing browser-native support

Caching & invalidation

- Why caching exists — trading correctness-freshness for speed
- Cache invalidation strategies, and why "cache invalidation is hard" is a real engineering problem, not a cliché

Message queues & event-driven systems

- Decoupling services with queues instead of direct calls
- At-least-once vs exactly-once delivery, and why that distinction matters

Load balancing & horizontal scaling

- Spreading load across multiple server instances
- Stateless vs stateful services, and why statelessness makes scaling easier

CAP theorem & replication

- Consistency, Availability, Partition tolerance — why you can't have all three
- Replication strategies and the trade-offs each makes under the CAP theorem

Capstone

Implement Raft (leader election) or build a load balancer — a project demonstrating real understanding of distributed-systems trade-offs, not just a CRUD app with extra steps.

This phase's foundation (routing, CRUD, validation, error handling, middleware) is what the more advanced topics above will build on directly.